From Lab to Edge: A Zero-Trust Architecture for Self-Hosted LLM Inference on a Raspberry Pi

Project Report

at the School of Business
in the study program Data Science and Artificial Intelligence

at the
DHBW Ravensburg

Author:	Max Mustermann
Module:	Methods and Tools of Software Engineering
Lecturer:	Prof. Dr.-Ing. Mark Schutera
Submission Date:	30.06.2099

Contents

1 Introduction **1**

2 Architecture, Tech-Stack and Added Value **2**

2.1 Small Language Model on Edge Devices 2

2.2 Network and Zero Trust Network Access 2

2.3 Separation of Responsibilities 3

3 Challenges and Architectual Solutions **4**

3.1 Ressource-Limitation in Edge-AI 4

3.2 Identity Loss in Zero Trust Networks 4

3.3 State Management in Distributed Systems 5

4 Conclusion, Limitations, and Outlook **6**

AI Usage Declaration **7**

Bibliography **8**

1 Introduction

The lecture Methods and Tools of Software Development teaches you all steps from working code, background services, version control, model integration, API-setup, databases, Reverse Proxy and finally deployment all along the PixelWise project. It is done by setting up laboratory conditions using two virtual machines (VMs). This isolation helps the learning experience by keeping problems contained, but leaves unanswered questions regarding real world implementations: Which ports do you expose when connecting your application to the internet? What if your application is limited by real world hardware? What trust do components need to communicate with one another? This project report is answering these questions. Instead of a production VM a real Raspberry Pi 4 with four Gigabyte of RAM and no open router-ports or traditional VPN as inference end point is used. The MNIST-image classifier is replaced by a quantized Small Language Model (SLM), running directly on the edge-device. This approach combines two modern arichitectural trends: placing AI-inference at the network edge because of cost effectiveness, latency reduction and data privacy compliance compared to centralized cloud architecture (vgl. Qu et al., 2025, pp. 1–3), as well as the replacement of perimeter security by Zero Trust Network Access (ZTNA), which authenticates every request individually and does not grant implicit trust based on network location (vgl. Mavroudis, 2024, 1f). This project report describes the building blocks this architecture is built upon. The final result can be used at: www.example.com.

2 Architecture, Tech-Stack and Added Value

The targeted architecture splits into three layers: a stateless front end in the global CDN, a ZTNA-based network layer without open entry ports and an on-premises hosted application layer that unites model, API and persistent data. Every one of those layers replaces a component used in the lecture with a more production suitable one.

2.1 Small Language Model on Edge Devices

The image classifier used in the lecture fits entirely into the memory of the dedicated VM and makes the end-to-end-data flow from the browser-canvas to the database entry transparent. Generative language models have fundamentally different resource requirements: even SLM with only a few billion parameters with their base floating-point representation reach beyond the memory limits of devices with four Gigabyte RAM (vgl. Seymour, Kutukcu, & Baidya, 2024, pp. 5–6). These models can only be sensibly implemented when post-training-quantized into a 4- or 8-bit-format (vgl. Husom et al., 2025, 4f). Quantization reduces the numerical precision of the model weights, thus also reducing the required memory capacity and the number of memory accesses per inference step at the cost of a measurable but acceptable accuracy loss (vgl. Husom et al., 2025, 4f). Specifically, the runtime environment Ollama was installed on the Raspberry Pi, which delivers quantized language models in a format optimized for edge inference. The FastAPI application communicates locally with this runtime, no input text leaves the device, every request has a predictable network latency, and no ongoing inference costs are incurred. These are precisely the advantages driving the edge AI paradigm (vgl. Qu et al., 2025, 1ff).

2.2 Network and Zero Trust Network Access

The lecture concludes the stack with a Nginx reverse proxy, which opens ports 80 and 443 on the production VM and forwards requests to the FastAPI application on localhost:8000. In a home network behind a NAT router without a dedicated public IP, this would require port forwarding or a dedicated VPN server.THis is precisely the form of perimeter security that proves problematic by modern standards. Zero Trust Network Access demands instead that every request is explicitly authenticated and authorized before reaching a protected resource (vgl. Mavroudis, 2024, p. 3). In practice, this approach is realized through reverse tunnel architectures: the internal device establishes an outbound, mutually authenticated connection to a globally distributed edge network, which subsequently routes inbound HTTP requests from the outside. For this purpose, the cloudflared

agent runs as a systemd service on the Raspberry Pi and maintains an outbound TLS connection to the Cloudflare edge nodes. An external request to the public subdomain is terminated in the Cloudflare network, checked against identity and WAF rules there, and only then forwarded to the backend through the existing tunnel. From the perspective of the home router, no inbound traffic flows at any point. From the perspective of the public internet, there is no reachable IP address, but merely the HTTPS endpoint in the Cloudflare network. Thus, the architecture eliminates the need for open ports, replaces the management of proprietary TLS certificates with automated edge termination, and shifts peak loads to a distributed network at the cost of an additional hop and a trust relationship with the tunnel provider.

2.3 Separation of Responsibilities

In the lecture, the frontend and backend are operated together on the production VM: Nginx serves static files from /var/www/pixelwise/ and proxies API requests to the FastAPI application on the same host. In the present architecture, this co-location is deliberately decoupled in order to satisfy the distinct requirement profiles of the two layers separately. The frontend, a Next.js application, is hosted on Vercel and delivered via their global CDN. Since the frontend maintains no application state, any request can be answered by any arbitrary CDN instance, a design principle described in microservice literature as a central lever for independent scalability and fault tolerance (vgl. Barua & Kaiser, 2025, 1f). On the Raspberry Pi, by contrast, the compute-intensive and stateful components remain: the FastAPI application as the API layer, the Ollama runtime with the loaded language model, and SQLite as persistent storage. This separation also allows both layers to be protected differently: the frontend benefits from the CDN platform's DDoS mitigation, while the backend remains completely hidden behind the ZTNA tunnel.

3 Challenges and Architectual Solutions

The transition from concept to working architecture revealed several challenges. Three challenges, each requiring an architectural response, will be discussed in detail. They range from the physical resource limitations of the edge device and the question of identity in the ZTNA layer, to the question of where the conversation state is authoritatively maintained.

3.1 Ressource-Limitation in Edge-AI

A Raspberry Pi 4 with four gigabytes of RAM immediately hits physical limits when running generative AI. With each additional conversation round, the context provided to the model grows, and the model allocates not only weight memory but also activation memory for inference. A chat history that is too long thus leads to an overflow of the main memory and the crash of the inference process (Out-of-Memory). The response lies on two levels. At the model level, a 4-bit quantization reduces the memory requirement of the weights by a factor of four compared to FP16, thereby creating the necessary headroom to keep the input and activations in RAM during inference in the first place (vgl. Husom et al., 2025, 4f). At the application level, a constant limit MAX_HISTORY_MESSAGES restricts the number of previous turns passed to the model. Older messages remain in the SQLite database for display purposes but are no longer included in the LLM context. This combined strategy of model compression and application-side resource management is the decisive lever that makes operating generative AI on a single-board computer reliable in the first place.

3.2 Identity Loss in Zero Trust Networks

To protect against spam and automated requests, the API was to receive IP-based rate limiting via SlowAPI. In its default configuration, SlowAPI uses the TCP source IP as the key. Right here, a peculiarity of the ZTNA architecture becomes apparent: since the Cloudflare Tunnel terminates the external connection at the edge and subsequently forwards the HTTP request to the FastAPI application via a local connection, 100 percent of all requests appear to originate from 127.0.0.1. The security mechanism was thus blind as a single limiter entry applied to all callers. The solution requires reconstructing identity one layer higher. ZTNA ss an identity-aware architecture in which the trust decision is no longer based on the network layer, but on application-layer attributes (vgl. Mavroudis, 2024, 4f). In practice, this is implemented through a customized key function in SlowAPI that reads the cf-connecting-ip HTTP header instead of the TCP source IP. Cloudflare

cryptographically verifies the real client IP at the edge and stamps it into precisely this header. Since the only permissible route to the backend runs through the authenticated tunnel, the header can be trusted. Such identity propagation across the trust boundary is described in recent studies on zero-trust microservice architectures as a necessary step for reconstructing reliable identity attributes (vgl. Rajendran et al., 2025, 1f).

3.3 State Management in Distributed Systems

In the first iteration, the frontend held the complete chat history and sent it to the backend as a JSON array with every request ("Frontend as Source of Truth"). This pattern is common because it keeps the backend stateless. In the LLM context, however, it creates two substantial problems. First, the HTTP payload grew linearly with each conversation round, which disadvantages mobile clients with unstable connections. Second, and more severely, an attacker could insert arbitrary text into the array as alleged assistant messages, effectively providing the model with an attacker-controlled history. Precisely this pattern is classified in the literature as prompt injection and is listed as one of the most severe security classes for LLM-integrated applications (vgl. Liu et al., 2025, 1f). The migration to "Backend as Source of Truth" solves both problems simultaneously. The endpoint now only accepts a `session_id` and the new user message. The FastAPI application loads the corresponding history from the SQLite database via SQLAlchemy, appends the new message, calls the model, and persists the response. Externally, the API remains stateless. Internally, it is stateful. A deliberate shift of state responsibility to the point where the data must be controlled anyway (vgl. Barua & Kaiser, 2025, 8f). The payload shrinks to a constant size, and the conversation history becomes inaccessible to client-side manipulation.

4 Conclusion, Limitations, and Outlook

The extension proves successful across all three design decisions. Generative AI can be operated reliably on a Raspberry Pi 4, provided that model compression and application-side resource management work in tandem. Public access functions without open router ports and without proprietary certificate management, and the separation of frontend and backend allows each layer to be operated in its respective suitable environment. However, the physical limitation of the edge device remains apparent. With a single active user, response times are within acceptable ranges, but as soon as multiple parallel conversations are conducted, the inference processes compete for the same four CPU cores, and the response time rises sharply. Productive scaling beyond the demonstration would require either dedicated inference accelerators (such as USB NPUs), batching at the model level, or a hybrid edge-cloud split, where the edge device answers simple requests locally and delegates more complex ones to a cloud endpoint. Within the chosen scope, however, the architecture fulfills exactly what it promises: a real, privacy-friendly, and low-cost demonstration of an edge AI application beyond the cloud paradigm. Beyond the purely technical observations, the project demonstrates just how far the gap between a controlled laboratory environment and productive operation actually extends. Each of the three extensions, edge inference, ZTNA, and decoupled frontend, closes a concrete gap in the stack taught in the course, without questioning its pedagogical value. On the contrary: it was the clean didactic reduction to two VMs that made the building blocks visible, which then had to be reassembled for a realistic edge deployment. The added value of the project report thus lies less in individual tooling decisions than in the consistent application of two current architectural principles: edge AI and zero trust, to a real, resource-constrained use case.

AI Usage Declaration

AI-Tool	**Usage**	**Application Field/Section**
Claude Opus	Translations, Text Rewrites, Frontend Coding	everywhere
Gemini Pro	Text Rewrites, Structure Checks, Frontend Coding	everywhere

Bibliography

Barua, B., & Kaiser, M. S. (2025). Design and Evaluation of a Microservices Cloud Framework for Online Travel Platforms. *arXiv.org*. Retrieved June 30, 2026, from https://arxiv.org/abs/2505.14508v1

Husom, E. J., Goknil, A., Astekin, M., Shar, L. K., Kåsen, A., Sen, S., Mithassel, B. A., & Soylu, A. (2025). Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency [arXiv:2504.03360 [cs.CY]]. https://doi.org/10.48550/arXiv.2504.03360
Comment: 30 pages, 14 figures.

Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., Zhang, L. Y., & Liu, Y. (2025). Prompt Injection attack against LLM-integrated Applications [arXiv:2306.05499 [cs.CR]]. https://doi.org/10.48550/arXiv.2306.05499

Mavroudis, V. (2024). Zero-Trust Network Access (ZTNA) [arXiv:2410.20611 [cs.CR]]. https://doi.org/10.48550/arXiv.2410.20611

Qu, G., Chen, Q., Wei, W., Lin, Z., Chen, X., & Huang, K. (2025). Mobile Edge Intelligence for Large Language Models: A Contemporary Survey [arXiv:2407.18921 [cs.NI]]. https://doi.org/10.48550/arXiv.2407.18921
Comment: 42 pages, 17 figures. This paper has been accepted by IEEE Communications Surveys & Tutorials.

Rajendran, R. N., Anumula, S. K., Rai, D. K., & Agrawal, S. (2025). Zero Trust Security Model Implementation in Microservices Architectures Using Identity Federation [arXiv:2511.04925 [cs.CR]].

Seymour, L., Kutukcu, B., & Baidya, S. (2024). Large Language Models on Small Resource-Constrained Systems: Performance Characterization, Analysis and Trade-offs [arXiv:2412.15352 [cs.LG]]. https://doi.org/10.48550/arXiv.2412.15352

Selbständigkeitserklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde. Des Weiteren versichere ich, dass die eingereichte elektronische Fassung mit der gedruckten Ausfertigung übereinstimmt.

,

Ort, Datum

Unterschrift